Intelligent IPC Chat Application

BTech Operating Systems Project Report

Real-time chat using Inter-Process Communication

Pipe | Message Queue | Shared Memory

Process management | Synchronization | ML analysis

Platform: Local Windows PC

Stack: React + FastAPI + Python multiprocessing

This document describes the project aim, features, architecture, OS concepts demonstrated, and how to run and present the system.

1. Project Aim

The aim of this project is to design and implement a complete, demonstration-ready Operating Systems application that shows how independent processes communicate on a real computer using Inter-Process Communication (IPC) mechanisms.

Unlike a frontend-only simulation, this application creates real OS processes (with real Process IDs), exchanges messages through Pipe, Message Queue, and Shared Memory, measures real latency, and displays live CPU/memory statistics. A polished control-center UI helps explain these concepts clearly during a viva or lab presentation.

2. Problem Statement

Operating Systems courses teach IPC, synchronization, and process management as abstract topics. Students often struggle to see how these ideas appear in real software. This project bridges that gap by building a chat-style application in which Alice, Bob, and Charlie are separate processes that talk through selectable IPC channels, while the user observes PIDs, latency, locks, and ML message analysis in real time.

3. Objectives

- Create and manage independent OS processes with real PIDs.
- Implement at least three IPC mechanisms: Pipe, Message Queue, Shared Memory.
- Demonstrate race conditions and safe synchronization using Mutex and Semaphore.
- Measure real IPC latency, throughput, CPU, and memory usage.
- Classify messages with a local machine-learning model (TF-IDF + Logistic Regression).
- Provide a professional UI suitable for academic demonstration.
- Persist messages, metrics, and experiment results in SQLite.

4. What the Project Does

The Intelligent IPC Chat Application is a local control center for inter-process messaging:

- Starts managed processes: IPC Broker, Alice, Bob, Charlie, and an ML Analyzer.
- Lets the user send chat messages from one process to another.
- Routes each message through the selected IPC mechanism.
- Shows delivery status, latency, message size, and mechanism used.
- Animates the path Alice -> IPC -> Bob on the visualizer.
- Displays live process stats (PID, CPU%, RAM, threads) via psutil.
- Runs OS experiments (race vs mutex, semaphore, producer-consumer, benchmarks).
- Classifies message text as normal, spam, toxic, or urgent using a local model.
- Stores history and experiment results in a local SQLite database.

Important note

The React frontend does not fake IPC. It only sends commands to the FastAPI backend. Actual communication happens between Python multiprocessing processes using OS-level IPC primitives.

5. Main Features

Chat System

Alice/Bob/Charlie messaging with timestamps, status, and inspector.

IPC Engine

Switchable Pipe, Message Queue, and Shared Memory channels.

Process Manager

Start, stop, restart managed processes; show real PIDs.

Synchronization Demos

Race condition, Mutex, Semaphore, Producer-Consumer.

IPC Monitoring Dashboard

CPU, RAM, latency, messages/sec, activity feed.

IPC Visualizer

Animated process -> mechanism -> process path.

IPC Comparison

Benchmark latency across mechanisms from real runs.

ML Message Analysis

TF-IDF + Logistic Regression classification + sentiment.

Anomaly Hints

Isolation Forest warnings on unusual resource patterns.

OS Experiments Panel

Ten runnable labs with live output and explanations.

Message History

SQLite-backed history of past transmissions.

OS Concepts Panel

Viva-ready definitions linked to project usage.

6. How It Works (Architecture)

High-level data flow:

```
  React UI  -->  FastAPI (REST + WebSocket)
                     |
                 IPC Manager
            /        |         \
      Alice      Broker      Bob / Charlie
            \        |         /
             Pipe / Queue / Shared Memory
                     |
                ML Analyzer Process
```

When the user clicks Send on the Chat page, the UI calls the backend API. The IPC Manager places a send command on the sender process control channel. The sender process writes the message using the selected IPC mechanism. The broker (for Pipe/Queue hub routing) or shared-memory path delivers it to the receiver. Events and measured latency are streamed back over WebSocket so the UI updates live.

6.1 Technology Stack

- Frontend: React, TypeScript, Vite, Tailwind CSS, Recharts, Framer Motion, Lucide.
- Backend: Python, FastAPI, Uvicorn, SQLAlchemy, SQLite.
- OS/IPC: multiprocessing.Pipe, Queue, SharedMemory, Lock, Semaphore; psutil.
- ML: scikit-learn (TF-IDF + Logistic Regression), Isolation Forest.

6.2 IPC Mechanisms

Pipe: Each worker has a duplex Pipe to the broker. Path: Alice <-> Pipe <-> Broker <-> Pipe <-> Bob.

Message Queue: Workers put messages into a multiprocessing.Queue. The broker forwards to receiver inboxes (classic producer-consumer).

Shared Memory: A SharedMemory buffer holds the payload. A Lock prevents races. A notify Queue wakes the peer (Windows-compatible wait/wake).

6.3 Synchronization

Race demo: multiple processes increment a shared counter without a lock, often producing a final count below the expected total (lost updates).

Mutex demo: the same increments protected by a Lock produce a consistent final count. Semaphore demo limits concurrent access (e.g., 2 permits).

6.4 Machine Learning

A small labeled dataset trains a local TF-IDF + Logistic Regression model to classify messages as normal, spam, toxic, or urgent. Sentiment is also estimated. Predictions run on the local machine - not a cloud API.

7. Application Modules (UI Menus)

Dashboard

Live CPU/RAM, latency, throughput, and activity feed.

Chat

Compose and send messages; watch live delivery and visualizer.

Processes

Table of managed processes with start/stop/restart.

IPC

Detailed path visualizer and per-mechanism metrics.

Comparison

Benchmark charts comparing Pipe vs Queue vs Shared Memory.

ML Analysis

Paste text and view classification, confidence, sentiment.

Experiments

Run OS labs with live results and explanations.

History

Persisted message history from SQLite.

Concepts

OS concept definitions for viva preparation.

Settings

Theme and connection status.

8. OS Concepts Demonstrated

Process

Independent program in execution with its own PID and address space.

IPC

Mechanisms that allow processes to exchange data safely.

Pipe

Byte stream channel between process endpoints.

Message Queue

Buffered producer-consumer messaging.

Shared Memory

Common memory region mapped into multiple processes.

Mutex / Lock

Mutual exclusion for critical sections.

Semaphore

Counting permit for limited concurrent access.

Race Condition

Incorrect results due to unsynchronized concurrent updates.

Producer-Consumer

Producers fill a buffer; consumers drain it.

Resource Monitoring

Observing CPU, memory, and threads of managed processes.

9. How to Run

Requirements: Windows PC, Python 3.11+, Node.js/npm.

Install (first time)

```
cd "...\OS IPC"
python -m venv .venv
.\.venv\Scripts\pip.exe install -r requirements.txt
cd frontend
npm install
```

Start backend (Terminal 1)

```
cd "...\OS IPC"
$env:PYTHONPATH = (Get-Location).Path
.\.venv\Scripts\python.exe -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

Do not use --reload on Windows with multiprocessing.

Start frontend (Terminal 2)

```
cd "...\OS IPC\frontend"
npm run dev
```

Open http://127.0.0.1:5173/ | API docs: http://127.0.0.1:8000/docs

10. Suggested Demo Flow (Viva)

1. Open Processes - show real PIDs matching Task Manager.
2. Open Chat - send Alice -> Bob using Pipe, then Queue, then Shared Memory.
3. Watch the IPC visualizer animate the message path.
4. Open Experiments - run Race Condition (unsafe count), then Mutex (safe count).
5. Open ML Analysis - classify a spam string vs a normal message.
6. Open Dashboard / Comparison - show measured latency charts.
7. Open Concepts - briefly define IPC, Mutex, and Shared Memory.

11. Results & Outcomes

- Real processes are created and managed with authentic OS PIDs.
- Messages are delivered through real IPC channels with measured latency.
- Race conditions produce inconsistent counters; locks restore consistency.
- Local ML model classifies messages without cloud dependency.
- The UI presents OS concepts in a form suitable for academic evaluation.

12. Limitations

- Runs locally on Windows; not a networked multi-machine chat system.
- Shared Memory uses a notify Queue for wait/wake (Windows-friendly design).
- ML dataset is educational-scale, not production spam filtering.
- Anomaly detection warns only; it does not auto-kill processes.

13. Future Enhancements

- Export experiment reports as PDF for lab submission.
- Named Windows pipe comparison panel.
- Stronger online anomaly training window.
- Multi-session chat rooms with persisted transcripts.

14. Conclusion

The Intelligent IPC Chat Application successfully demonstrates core Operating Systems concepts-process management, IPC, synchronization, monitoring, and lightweight ML-on a real Windows PC. By combining a professional control-center UI with genuine multiprocessing IPC, the project makes abstract OS theory concrete, measurable, and easy to present in a viva.

End of report. Source code: backend/ (FastAPI + IPC runtime), frontend/ (React UI), docs/os-concepts.md (viva notes).